Part 4

Full REST API for All Tables

Products · Offices · Employees · Orders · Payments · OrderDetails · ProductLines

Building on the Customer API from Part 2 — same 4-layer architecture, same logging principles, extended to cover every table in the database with complete CRUD operations.

Products	Offices	Employees	Orders	Payments	ProductLines

Overview

What You Are Building in Part 4

In Part 2, you built a complete Customer API with the 4-layer architecture (database.py, schemas.py, crud.py, router.py). Part 4 extends that same system to cover every remaining table in the database. You do not start over — you extend.

Architecture Reminder

Every new table follows the exact same pattern you already learned:

The 4 Layers (same as Part 2)

1. database.py — Already done. No changes needed.
2. schemas.py — Add new Pydantic models for each new table.
3. crud.py — Add new CRUD functions for each new table.
4. router.py — Add new routers for each new table, then register them in main.py.

Tables You Will Build APIs For

Table	Router File	CRUD File	Schema Models
`products`	products_router.py	products_crud.py	ProductCreate/Out/Update
`productlines`	productlines_router.py	productlines_crud.py	ProductLineCreate/Out/Update
`offices`	offices_router.py	offices_crud.py	OfficeCreate/Out/Update
`employees`	employees_router.py	employees_crud.py	EmployeeCreate/Out/Update
`orders`	orders_router.py	orders_crud.py	OrderCreate/Out/Update
`orderdetails`	orderdetails_router.py	orderdetails_crud.py	OrderDetailCreate/Out/Update
`payments`	payments_router.py	payments_crud.py	PaymentCreate/Out/Update

Project File Structure After Part 4

Your project folder should look like this:

```
project/
    ├── main.py
    ├── database.py
    ├── logger.py
    ├── .env
    ├── requirements.txt
```

```
├── schemas/
│   ├── customer_schemas.py    ← already done
│   ├── product_schemas.py
│   ├── productline_schemas.py
│   ├── office_schemas.py
│   ├── employee_schemas.py
│   ├── order_schemas.py
│   ├── orderdetail_schemas.py
│   └── payment_schemas.py
├── crud/
│   ├── customer_crud.py       ← already done
│   ├── product_crud.py
│   ├── productline_crud.py
│   ├── office_crud.py
│   ├── employee_crud.py
│   ├── order_crud.py
│   ├── orderdetail_crud.py
│   └── payment_crud.py
└── routers/
    ├── customer_router.py     ← already done
    ├── product_router.py
    ├── productline_router.py
    ├── office_router.py
    ├── employee_router.py
    ├── order_router.py
    ├── orderdetail_router.py
    └── payment_router.py
```

Products

Managing the product catalog with stock and pricing

Step 1 — Define the Schemas (schemas.py)

You need three Pydantic models for the products resource. Add these to your schemas file.

Schema Checklist

ProductCreate — Fields needed to create a new record. Primary key is excluded if auto-generated.
ProductOut — What the API returns to the user. Includes the primary key and any related data.
ProductUpdate — Same fields as Create but every field is Optional (for partial updates).

Fields and Validation Rules

Field	Type	Required	Notes
productCode	str	Yes	Primary key. Up to 15 characters. e.g. 'S10_1678'
productName	str	Yes	Full product name. Max 70 characters.
productLine	str	Yes	Foreign key → productlines.productLine
productScale	str	Yes	Scale of the model. e.g. '1:10'
productVendor	str	Yes	Vendor name. Max 50 characters.
productDescription	str	Yes	Full text description of the product.
quantityInStock	int	Yes	Must be >= 0. Whole numbers only.
buyPrice	Decimal(10,2)	Yes	Cost price. Two decimal places.
MSRP	Decimal(10,2)	Yes	Retail price. Must be >= buyPrice.

Example: ProductCreate (schemas.py)

```
class ProductCreate(BaseModel):
    productCode: str
    productName: str
    productLine: str
    productScale: str
    productVendor: str
    productDescription: str
    quantityInStock: int
    buyPrice: Decimal
    MSRP: Decimal
```

```
class ProductOut(ProductCreate):
    class Config:
        from_attributes = True

class ProductUpdate(BaseModel):
    productName: Optional[str] = None
    productLine: Optional[str] = None
    quantityInStock: Optional[int] = None
    buyPrice: Optional[Decimal] = None
    MSRP: Optional[Decimal] = None
```

Step 2 — Write CRUD Functions (crud.py)

Create one function per operation. Each function takes the database session as its first parameter. Never call external services — only interact with the database.

CRUD Functions to Implement

```
get_productss(db, skip, limit)          → List all records with pagination
get_products(db, id)                    → Get single record by primary key
create_products(db, data)               → Insert a new record
update_products(db, id, data)           → Update an existing record (partial)
delete_products(db, id)                 → Delete a record by primary key
get_products_with_orderdetails(db, id)  → Fetch record with related orderdetails
```

Error Handling Rules:

- If a record is not found, raise an HTTPException with status_code=404.
- If a foreign key constraint fails (e.g., invalid productLine), return a 422 error with a clear message.
- Log every operation: what was requested, what was returned, and any errors.

Step 3 — Create the Router (router.py)

Create a new file: routers/products_router.py. Register it in main.py with the prefix shown below.

Method	Endpoint	Description
GET	/products	List all products. Supports skip and limit for pagination.
GET	/products/{productCode}	Get a single product by its product code. Returns 404 if not found.
GET	/products/{productCode}/orderdetails	Get a product with all its order line details.
POST	/products	Create a new product. Requires all fields. Validates productLine FK.

Method	Endpoint	Description
PUT	/products/{productCode}	Update a product by code. All fields optional (partial update).
DELETE	/products/{productCode}	Delete a product. Returns 404 if not found.

Key Notes for This Router

productCode is a string primary key (not auto-incremented). The client provides it on creation.
buyPrice and MSRP must be stored as Decimal, not float, to avoid rounding errors.
If a productLine foreign key does not exist in the productlines table, return 422 with a helpful message.
The GET /products/{productCode}/orderdetails endpoint should return an empty list [] if no orders exist.

ProductLines

Product category management

Step 1 — Define the Schemas (schemas.py)

You need three Pydantic models for the productlines resource. Add these to your schemas file.

Schema Checklist

ProductLineCreate — Fields needed to create a new record. Primary key is excluded if auto-generated.
ProductLineOut — What the API returns to the user. Includes the primary key and any related data.
ProductLineUpdate — Same fields as Create but every field is Optional (for partial updates).

Fields and Validation Rules

Field	Type	Required	Notes
productLine	str	Yes	Primary key. Category name. Max 50 characters.
textDescription	str	No	Plain text description. Max 4000 characters.
htmlDescription	str	No	HTML version of the description. Optional.
image	bytes	No	Binary image data. Optional. May be null.

Step 2 — Write CRUD Functions (crud.py)

Create one function per operation. Each function takes the database session as its first parameter. Never call external services — only interact with the database.

CRUD Functions to Implement

get_productliness(db, skip, limit) → List all records with pagination
get_productlines(db, id) → Get single record by primary key
create_productlines(db, data) → Insert a new record
update_productlines(db, id, data) → Update an existing record (partial)
delete_productlines(db, id) → Delete a record by primary key
get_productlines_with_products(db, id) → Fetch record with related products

Error Handling Rules:

- If a record is not found, raise an HTTPException with status_code=404.
- If a foreign key constraint fails (e.g., invalid productLine), return a 422 error with a clear message.

- Log every operation: what was requested, what was returned, and any errors.

Step 3 — Create the Router (router.py)

Create a new file: routers/productlines_router.py. Register it in main.py with the prefix shown below.

Method	Endpoint	Description
GET	/productlines	List all product lines. Supports pagination.
GET	/productlines/{productLine}	Get a single product line by name. 404 if not found.
GET	/productlines/{productLine}/products	Get a product line with all its products listed.
POST	/productlines	Create a new product line. Only productLine is required.
PUT	/productlines/{productLine}	Update a product line. All fields optional.
DELETE	/productlines/{productLine}	Delete a product line. Will fail if products still reference it.

Key Notes for This Router

productLine is a string primary key — the client must provide it.
image is binary (BYTEA in PostgreSQL). Handle it carefully; do not return raw bytes in JSON. You may choose to exclude it from ProductOut or encode it as base64.
Deleting a productLine that still has products referencing it will cause a foreign key constraint error. Catch this and return a 409 Conflict error with a clear message.

Offices

Company office locations around the world

Step 1 — Define the Schemas (schemas.py)

You need three Pydantic models for the offices resource. Add these to your schemas file.

Schema Checklist

OfficeCreate — Fields needed to create a new record. Primary key is excluded if auto-generated.
OfficeOut — What the API returns to the user. Includes the primary key and any related data.
OfficeUpdate — Same fields as Create but every field is Optional (for partial updates).

Fields and Validation Rules

Field	Type	Required	Notes
officeCode	str	Yes	Primary key. Short code. e.g. '1', '2', '7'.
city	str	Yes	City name. Max 50 characters.
phone	str	Yes	Phone number including country code.
addressLine1	str	Yes	Main address line. Max 50 characters.
addressLine2	str	No	Optional second address line.
state	str	No	State or region. Optional. May be null.
country	str	Yes	Country name. Max 50 characters.
postalCode	str	Yes	Postal or ZIP code. Max 15 characters.
territory	str	Yes	Sales territory. e.g. 'NA', 'EMEA', 'APAC', 'Japan'.

Step 2 — Write CRUD Functions (crud.py)

Create one function per operation. Each function takes the database session as its first parameter. Never call external services — only interact with the database.

CRUD Functions to Implement

get_officess(db, skip, limit) → List all records with pagination
get_offices(db, id) → Get single record by primary key
create_offices(db, data) → Insert a new record
update_offices(db, id, data) → Update an existing record (partial)
delete_offices(db, id) → Delete a record by primary key

get_offices_with_employees(db, id) → Fetch record with related employees

Error Handling Rules:

- If a record is not found, raise an HTTPException with status_code=404.
- If a foreign key constraint fails (e.g., invalid productLine), return a 422 error with a clear message.
- Log every operation: what was requested, what was returned, and any errors.

Step 3 — Create the Router (router.py)

Create a new file: routers/offices_router.py. Register it in main.py with the prefix shown below.

Method	Endpoint	Description
GET	/offices	List all offices. Supports pagination.
GET	/offices/{officeCode}	Get a single office by office code. 404 if not found.
GET	/offices/{officeCode}/employees	Get an office with all employees who work there.
POST	/offices	Create a new office. All required fields must be present.
PUT	/offices/{officeCode}	Update an office. All fields optional (partial update).
DELETE	/offices/{officeCode}	Delete an office. Fails if employees still reference it.

Key Notes for This Router

officeCode is a string primary key. The client provides it.
state is nullable. Your schema must allow None / null for this field.
If you try to delete an office that still has employees, catch the FK constraint error and return 409.
The GET with employees endpoint should return an empty list [] if no employees are assigned to that office.

Employees

Staff records with self-referencing management hierarchy

Step 1 — Define the Schemas (schemas.py)

You need three Pydantic models for the employees resource. Add these to your schemas file.

Schema Checklist

EmployeeCreate — Fields needed to create a new record. Primary key is excluded if auto-generated.
EmployeeOut — What the API returns to the user. Includes the primary key and any related data.
EmployeeUpdate — Same fields as Create but every field is Optional (for partial updates).

Fields and Validation Rules

Field	Type	Required	Notes
employeeNumber	int	Yes	Primary key. Unique integer. Provided by client.
lastName	str	Yes	Employee last name. Max 50 characters.
firstName	str	Yes	Employee first name. Max 50 characters.
extension	str	Yes	Phone extension. e.g. 'x5800'.
email	str	Yes	Must be a valid email format.
officeCode	str	Yes	Foreign key → offices.officeCode.
reportsTo	int	No	FK → employees.employeeNumber. Null for the top manager.
jobTitle	str	Yes	Job title. e.g. 'Sales Rep', 'VP Sales'.

Step 2 — Write CRUD Functions (crud.py)

Create one function per operation. Each function takes the database session as its first parameter. Never call external services — only interact with the database.

CRUD Functions to Implement

get_employeess(db, skip, limit) → List all records with pagination
get_employees(db, id) → Get single record by primary key
create_employees(db, data) → Insert a new record
update_employees(db, id, data) → Update an existing record (partial)
delete_employees(db, id) → Delete a record by primary key
get_employees_with_customers(db, id) → Fetch record with related customers

Error Handling Rules:

- If a record is not found, raise an HTTPException with status_code=404.
- If a foreign key constraint fails (e.g., invalid productLine), return a 422 error with a clear message.
- Log every operation: what was requested, what was returned, and any errors.

Step 3 — Create the Router (router.py)

Create a new file: routers/employees_router.py. Register it in main.py with the prefix shown below.

Method	Endpoint	Description
GET	/employees	List all employees with pagination.
GET	/employees/{employeeNumber}	Get single employee by number. 404 if not found.
GET	/employees/{employeeNumber}/customers	Get employee with the list of customers they manage.
GET	/employees/{employeeNumber}/reports	Get all employees who report to this employee.
POST	/employees	Create a new employee. Validate officeCode and reportsTo FKs.
PUT	/employees/{employeeNumber}	Update an employee. All fields optional.
DELETE	/employees/{employeeNumber}	Delete an employee. Fails if they have direct reports or customers.

Key Notes for This Router

reportsTo is a self-referencing foreign key — an employee references another employee. This is nullable (the top-level manager has no manager above them).
email should be validated as a proper email string using Pydantic's EmailStr type (pip install email-validator).
The /reports endpoint returns all employees whose reportsTo equals this employee's number.
Deleting an employee who has direct reports or who is assigned as a sales rep to customers should return 409.

Orders

Customer order tracking with status and dates

Step 1 — Define the Schemas (schemas.py)

You need three Pydantic models for the orders resource. Add these to your schemas file.

Schema Checklist

OrderCreate — Fields needed to create a new record. Primary key is excluded if auto-generated.
OrderOut — What the API returns to the user. Includes the primary key and any related data.
OrderUpdate — Same fields as Create but every field is Optional (for partial updates).

Fields and Validation Rules

Field	Type	Required	Notes
orderNumber	int	Yes	Primary key. Auto-assigned or provided by client.
orderDate	date	Yes	Date the order was placed. Format: YYYY-MM-DD.
requiredDate	date	Yes	Date by which the order must be delivered.
shippedDate	date	No	Actual ship date. Null if not yet shipped.
status	str	Yes	Must be one of: Shipped, Resolved, Cancelled, On Hold, Disputed, In Process.
comments	str	No	Free-text notes about the order. Optional.
customerNumber	int	Yes	Foreign key → customers.customerNumber.

Step 2 — Write CRUD Functions (crud.py)

Create one function per operation. Each function takes the database session as its first parameter. Never call external services — only interact with the database.

CRUD Functions to Implement

get_orderss(db, skip, limit) → List all records with pagination
get_orders(db, id) → Get single record by primary key
create_orders(db, data) → Insert a new record
update_orders(db, id, data) → Update an existing record (partial)
delete_orders(db, id) → Delete a record by primary key
get_orders_with_orderdetails(db, id) → Fetch record with related orderdetails

Error Handling Rules:

- If a record is not found, raise an HTTPException with status_code=404.
- If a foreign key constraint fails (e.g., invalid productLine), return a 422 error with a clear message.
- Log every operation: what was requested, what was returned, and any errors.

Step 3 — Create the Router (router.py)

Create a new file: routers/orders_router.py. Register it in main.py with the prefix shown below.

Method	Endpoint	Description
GET	/orders	List all orders with pagination.
GET	/orders/{orderNumber}	Get a single order by number. 404 if not found.
GET	/orders/{orderNumber}/orderdetails	Get an order with all its line items (products ordered).
GET	/orders/customer/{customerNumber}	Get all orders for a specific customer.
POST	/orders	Create a new order. Validate customerNumber FK.
PUT	/orders/{orderNumber}	Update an order. Useful for updating status or shippedDate.
DELETE	/orders/{orderNumber}	Delete an order. Will fail if orderdetails still reference it.

Key Notes for This Router

status must be validated with a Literal type or Enum in Pydantic. Only the 6 listed values are valid.
shippedDate is nullable. Only set it when the order is actually shipped.
requiredDate should be validated to be after orderDate. Add a Pydantic validator for this.
The /customer/{customerNumber} endpoint returns [] if the customer has no orders — never raise a 404 for this.
Deleting an order that still has orderdetails rows should return a 409 Conflict, not a 500 error.

OrderDetails

Individual line items within each order

Step 1 — Define the Schemas (schemas.py)

You need three Pydantic models for the orderdetails resource. Add these to your schemas file.

Schema Checklist

OrderDetailCreate — Fields needed to create a new record. Primary key is excluded if auto-generated.
OrderDetailOut — What the API returns to the user. Includes the primary key and any related data.
OrderDetailUpdate — Same fields as Create but every field is Optional (for partial updates).

Fields and Validation Rules

Field	Type	Required	Notes
orderNumber	int	Yes	Part of composite PK. FK → orders.orderNumber.
productCode	str	Yes	Part of composite PK. FK → products.productCode.
quantityOrdered	int	Yes	Quantity of this product ordered. Must be > 0.
priceEach	Decimal(10,2)	Yes	Price per unit at time of order. Two decimal places.
orderLineNumber	int (smallint)	Yes	Line number within the order. For ordering/sorting the items.

Step 2 — Write CRUD Functions (crud.py)

Create one function per operation. Each function takes the database session as its first parameter. Never call external services — only interact with the database.

CRUD Functions to Implement

get_orderdetailss(db, skip, limit) → List all records with pagination
get_orderdetails(db, id) → Get single record by primary key
create_orderdetails(db, data) → Insert a new record
update_orderdetails(db, id, data) → Update an existing record (partial)
delete_orderdetails(db, id) → Delete a record by primary key

Error Handling Rules:

- If a record is not found, raise an HTTPException with status_code=404.

- If a foreign key constraint fails (e.g., invalid productLine), return a 422 error with a clear message.
- Log every operation: what was requested, what was returned, and any errors.

Step 3 — Create the Router (router.py)

Create a new file: routers/orderdetails_router.py. Register it in main.py with the prefix shown below.

Method	Endpoint	Description
GET	/orderdetails	List all order detail records with pagination.
GET	/orderdetails/{orderNumber}/{productCode}	Get a single line item by composite key. 404 if not found.
GET	/orderdetails/order/{orderNumber}	Get all line items for a specific order.
GET	/orderdetails/product/{productCode}	Get all orders that contain a specific product.
POST	/orderdetails	Add a new line item. Validates both orderNumber and productCode FKs.
PUT	/orderdetails/{orderNumber}/{productCode}	Update quantity or price. All fields optional.
DELETE	/orderdetails/{orderNumber}/{productCode}	Remove a line item from an order.

Key Notes for This Router

The primary key is COMPOSITE: (orderNumber, productCode). Both are needed for GET, PUT, and DELETE operations.
Your CRUD functions must accept both keys as parameters, e.g. get_orderdetail(db, order_number, product_code).
priceEach stores the price at the time of purchase — it may differ from the current product MSRP.
quantityOrdered must be validated as a positive integer (> 0). Zero or negative quantities are invalid.
orderLineNumber is a smallint. Validate it fits within the smallint range (1–32,767).

Payments

Customer payment records linked by check number

Step 1 — Define the Schemas (schemas.py)

You need three Pydantic models for the payments resource. Add these to your schemas file.

Schema Checklist

PaymentCreate — Fields needed to create a new record. Primary key is excluded if auto-generated.
PaymentOut — What the API returns to the user. Includes the primary key and any related data.
PaymentUpdate — Same fields as Create but every field is Optional (for partial updates).

Fields and Validation Rules

Field	Type	Required	Notes
customerNumber	int	**Yes**	Part of composite PK. FK → customers.customerNumber.
checkNumber	str	**Yes**	Part of composite PK. Unique check identifier. Max 50 chars.
paymentDate	date	**Yes**	Date the payment was made. Format: YYYY-MM-DD.
amount	Decimal(10,2)	**Yes**	Payment amount. Must be > 0. Two decimal places.

Step 2 — Write CRUD Functions (crud.py)

Create one function per operation. Each function takes the database session as its first parameter. Never call external services — only interact with the database.

CRUD Functions to Implement

get_paymentss(db, skip, limit) → List all records with pagination
get_payments(db, id) → Get single record by primary key
create_payments(db, data) → Insert a new record
update_payments(db, id, data) → Update an existing record (partial)
delete_payments(db, id) → Delete a record by primary key

Error Handling Rules:

- If a record is not found, raise an HTTPException with status_code=404.
- If a foreign key constraint fails (e.g., invalid productLine), return a 422 error with a clear message.

- Log every operation: what was requested, what was returned, and any errors.

Step 3 — Create the Router (router.py)

Create a new file: routers/payments_router.py. Register it in main.py with the prefix shown below.

Method	Endpoint	Description
GET	/payments	List all payment records with pagination.
GET	/payments/{customerNumber}/{checkNumber}	Get a single payment by composite key. 404 if not found.
GET	/payments/customer/{customerNumber}	Get all payments made by a specific customer.
POST	/payments	Record a new payment. Validates customerNumber FK.
PUT	/payments/{customerNumber}/{checkNumber}	Update payment date or amount. All fields optional.
DELETE	/payments/{customerNumber}/{checkNumber}	Delete a payment record.

Key Notes for This Router

Like orderdetails, the primary key is COMPOSITE: (customerNumber, checkNumber).
amount must be validated as strictly positive (> 0). Zero or negative payments are invalid.
paymentDate should not be in the future. Add a Pydantic validator using datetime.date.today().
The GET /payments/customer/{customerNumber} endpoint returns [] if the customer has no payments — do not raise 404.
checkNumber is a string, not an integer. It represents a bank check number and may contain letters.

Final Step — Register All Routers in main.py

Connecting every router to your FastAPI application

After creating all the routers, you must register them in your main.py file. This is what makes the endpoints available when you start the server.

main.py — Complete Registration Code

main.py

```
from fastapi import FastAPI
from routers import (
    customer_router,
    product_router,
    productline_router,
    office_router,
    employee_router,
    order_router,
    orderdetail_router,
    payment_router,
)
from database import engine, Base
from logger import get_logger

logger = get_logger(__name__)
app = FastAPI(title='ClassicModels API', version='2.0')

app.include_router(customer_router.router,     prefix='/customers',     tags=['Customers'])
app.include_router(product_router.router,      prefix='/products',      tags=['Products'])
app.include_router(productline_router.router,  prefix='/productlines',  tags=['ProductLines'])
app.include_router(office_router.router,       prefix='/offices',       tags=['Offices'])
app.include_router(employee_router.router,     prefix='/employees',     tags=['Employees'])
app.include_router(order_router.router,        prefix='/orders',        tags=['Orders'])
app.include_router(orderdetail_router.router,  prefix='/orderdetails',  tags=['OrderDetails'])
app.include_router(payment_router.router,      prefix='/payments',      tags=['Payments'])

@app.get('/')
def root():
    logger.info('Root endpoint accessed')
    return {'message': 'ClassicModels API is running!'}
```

Testing Your API

Once main.py is set up and all routers are registered, test the full API:

- Start your server: uvicorn main:app --reload
- Open http://localhost:8000/docs to see all endpoints in the interactive Swagger UI.

- Every table should appear as its own tagged section in the docs.
- Test at least one GET, POST, PUT, and DELETE for each resource before submitting.

Logging Checklist (All Layers)

Layer	What to Log
`router.py`	Incoming request (method + path + params). Response status. 404 / 500 errors.
`crud.py`	Each query executed. Records found or not found. Create / update / delete confirmations.
`schemas.py`	Validation errors with field name and received value.
`database.py`	Connection established. Connection closed. Any connection errors.